



Chair of
Information Theory
and Data Analytics

RWTHAACHEN
UNIVERSITY

RESOURCE-EFFICIENT INFERENCE IN LARGE GENERATIVE MODELS

PATRICK TAZO KUETE

A seminar submitted to the
Faculty of Electrical Engineering and Information Technology,
RWTH Aachen University,
supervised by Hazem el Hadj Khalifa, M.Sc.

CHAIR OF INFORMATION THEORY AND DATA ANALYTICS
RWTH AACHEN UNIVERSITY

Abstract

Large generative models, in particular decoder-only Transformer architectures, have demonstrated unprecedented capabilities in natural language processing, code synthesis, and multimodal reasoning. However, the inference process is computationally intensive: repeated autoregressive forward passes impose severe demands on memory, compute, memory bandwidth, and energy. These characteristics make deployment on resource-constrained edge devices — such as smartphones, embedded systems, and Internet of Things (IoT) hardware — a significant open challenge.

This paper provides a structured survey of resource-efficient inference in large generative models. Starting from the fundamental Transformer building blocks, it traces the complete inference pipeline and systematically identifies the principal resource bottlenecks at each stage, distinguishing between the compute-bound prefill phase and the memory-bandwidth-bound decode phase. Building on this analysis, the paper surveys the leading efficiency techniques across four layers: model compression (quantisation, pruning, and knowledge distillation), inference-level algorithmic improvements (optimised attention mechanisms and Key-Value (KV) cache management), decoding strategies (speculative decoding and batching), and system-level runtimes (vLLM, llama.cpp, and MLC-LLM). For each technique, the discussion focuses on the specific bottleneck it addresses and the practical trade-offs that arise when targeting edge hardware.

Contents

List of Abbreviations	xi
1 Introduction	1
2 Preliminaries and Taxonomy	3
2.1 Transformer Architecture and Inference Process in Large Language Models	3
2.2 The Autoregressive Inference Pipeline	9
2.3 Step-by-Step Bottleneck Analysis	11
2.3.1 Step 1: Tokenisation	11
2.3.2 Step 2: Token Embedding and Positional Encoding	11
2.3.3 Step 3: Multi-Head Self-Attention	11
2.3.4 Step 4: Feed-Forward Network	12
2.3.5 Step 5: LM Head and Sampling	12
2.4 Limitations Beyond Inference	12
2.5 Taxonomy and Key Metrics	13
3 Towards Efficient Inference: Techniques and Solutions	15
3.1 Model Compression	16
3.1.1 Quantisation	16
3.1.2 Pruning	17
3.1.3 Knowledge Distillation	19
3.2 Inference-Level Optimisations	19
3.2.1 Multi-Query and Grouped-Query Attention	20
3.2.2 FlashAttention	20
3.2.3 KV Cache Optimisation	21
3.3 Decoding Strategies	21
3.3.1 Speculative Decoding	21
3.3.2 Continuous Batching	22
3.4 System-Level Runtimes and Edge Frameworks	22
3.4.1 llama.cpp	23
3.4.2 MLC-LLM	23
3.4.3 vLLM	24
3.5 Discussion: Trade-offs and Edge-Specific Considerations	24
3.5.1 The Efficiency–Accuracy Trade-off	24
3.5.2 Orthogonality and Composability	24

Contents

3.5.3 The Fundamental Edge Constraint: Memory	25
4 Conclusion	27
Bibliography	29

List of Figures

- 2.1 Schematic of the Transformer inference pipeline. Red-shaded steps (3, 4) represent the primary resource bottlenecks. The decode loop repeats steps 3–5 until an end-of-sequence token is produced. 10
- 3.1 Mapping of inference bottlenecks (red) to the efficiency techniques (blue) surveyed in this chapter. Arrows indicate which technique targets which bottleneck; several techniques address multiple bottlenecks simultaneously. . 15

List of Tables

List of Abbreviations

AI Artificial Intelligence 1

BLAS Basic Linear Algebra Subprograms 13

CPU Central Processing Unit 13

DRAM Dynamic Random Access Memory 7, 21, 23

FFN Feed-Forward Network 4, 7, 14, 23

FLOP Floating Point Operation 13

FP16 16-bit Floating Point 12, 13, 16, 23

GPU Graphics Processing Unit 13, 16, 17, 19

GQA Grouped-Query Attention 8, 16, 21, 23

HBM High-Bandwidth Memory 16

INT4 4-bit Integer 12

INT8 8-bit Integer 12, 13, 17

IoT Internet of Things iii

KD Knowledge Distillation 15

KV Key-Value iii, 4–8, 14–17, 19–21, 23

LLM Large Language Model 1, 3, 4, 11–14, 19, 23

LoRA Low-Rank Adaptation 14

MFU Model FLOP Utilisation 7

MHA Multi-Head Attention 3, 16

MLC Machine Learning Compilation iii, 19

MLP Multilayer Perceptron 4

MQA Multi-Query Attention 8, 16

PTQ Post-Training Quantisation 12, 13

QAT Quantisation-Aware Training 13

RoPE Rotary Positional Encoding 3

SRAM Static Random Access Memory 16

TPOT Time per Output Token 9

TTFT Time to First Token 9

1 Introduction

The past decade has witnessed a remarkable transformation in Artificial Intelligence (AI) research, driven by the emergence and rapid scaling of large generative models. Systems such as GPT-3 [4], LLaMA [17], and their successors have demonstrated near-human performance on a broad spectrum of tasks—from natural language understanding and code generation to mathematical reasoning and multimodal dialogue. This progress rests fundamentally on the Transformer architecture [18], which, when trained on trillions of tokens and scaled to hundreds of billions of parameters, exhibits surprising emergent capabilities.

Yet capability has come at a steep cost. Generating even a single response with a state-of-the-art Large Language Model (LLM) requires billions of floating-point operations, gigabytes of memory, and significant electrical power. In data-centre settings this cost is absorbed through specialised hardware and economies of scale, but it nonetheless drives substantial economic and environmental expenditure. On resource-constrained platforms—smartphones, embedded controllers, autonomous vehicles—such demands can make deployment entirely infeasible without dedicated optimisation. The field of resource-efficient LLM inference has therefore emerged as one of the most active and practically consequential research areas in machine learning [3].

This paper provides a structured survey of the state of the art in resource-efficient inference for large generative models. It begins by grounding the analysis in a concrete understanding of what makes the present day Large Generative Models so powerful—Transformers—and goes on to briefly explain the process of inference, highlighting where applicable, which resources are the limiting factor. In addition to this, a short list of terms deemed important will be defined, to give a clear foundation (Chapter chapter 2). As the main course of this Paper, a comprehensive survey of efficiency techniques follows in chapter 3. This survey is structured around the three main axes of efficiency: model compression, algorithmic optimisation, and hardware acceleration. Each technique is described in terms of its underlying principles, practical implementation considerations, and empirical performance trade-offs. Finally, the paper concludes with a discussion of open challenges and future directions in the field chapter 4.

2 Preliminaries and Taxonomy

2.1 Transformer Architecture and Inference Process in Large Language Models

Large Language Models (LLMs) are predominantly built on the Transformer architecture, which has become the central neural architecture for modern natural language processing and text generation systems. The reviewed papers describe the Transformer not merely as a sequence-processing model, but as the architectural foundation that enables large-scale language models to represent contextual information, generate text autoregressively, and support a wide range of downstream language tasks [3], [19]. In current LLMs, especially decoder-only models such as GPT-style systems, the Transformer architecture is closely coupled with an inference procedure based on next-token prediction. This procedure is effective for open-ended generation, but it also introduces major computational and memory bottlenecks, particularly because attention scales with sequence length and because decoding proceeds token by token [19], [3].

2.1.1 Transformer Architecture

The Transformer processes language by first converting discrete text into numerical representations. In a typical LLM pipeline, a tokenizer maps the original input sentence or prompt into a sequence of tokens. These tokens are then passed through an embedding layer, which transforms each token into a continuous vector representation. Since the Transformer does not use recurrence or convolution to encode order, positional information must be added to the token embeddings. Positional encodings or positional embeddings provide the model with information about the relative or absolute location of tokens in the sequence, allowing the architecture to distinguish between otherwise identical tokens occurring at different positions [3], [19].

After tokenization and embedding, the sequence is processed by a stack of Transformer blocks. The papers describe the standard Transformer block as consisting mainly of a multi-head self-attention module, a feed-forward network, and normalization operations [19]. In decoder-based LLMs, these blocks are arranged sequentially, so the hidden representation produced by one block becomes the input to the next. At the end of the stack,

a prediction head maps the final hidden representation to a probability distribution over the vocabulary, from which the next token can be selected [3].

The core operation of the Transformer is self-attention. Self-attention allows the model to compute contextual representations by relating each token to other tokens in the sequence. For an input representation X , the model learns separate linear projections that produce queries Q , keys K , and values V . The attention output is computed by comparing queries with keys, normalizing the resulting similarity scores, and using these weights to aggregate values:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V,$$

where d_k denotes the dimensionality of the key vectors. This formulation enables each token representation to selectively incorporate information from other tokens. The use of the dot product QK^T makes the operation sensitive to semantic and syntactic relationships between tokens, while the softmax function converts the similarity scores into normalized attention weights [3], [19].

In practical Transformer models, self-attention is extended to multi-head self-attention. Instead of computing a single attention distribution, the model performs multiple attention operations in parallel using different learned projections. Each head can attend to different relations in the input, and the outputs of all heads are concatenated and projected back into the model’s hidden dimension. This design increases the representational capacity of the model because different heads may capture different dependencies, such as local syntactic relations, long-range semantic relations, or task-specific contextual cues [3], [19].

The feed-forward network is the second major component of the Transformer block. It is usually positioned after the attention module and consists of two linear transformations separated by a nonlinear activation function. If X denotes the input to this module, the feed-forward computation can be written as:

$$\text{FFN}(X) = W_2 \sigma(W_1 X), \tag{2.1.1}$$

where W_1 and W_2 are learned weight matrices and σ is a nonlinear activation function [19]. While self-attention mixes information across token positions, the feed-forward network transforms each token representation independently in the feature dimension. Together, attention and feed-forward layers allow the Transformer to combine sequence-level contextualization with high-dimensional nonlinear feature transformation.

The reviewed sources distinguish between encoder-only, encoder-decoder, and decoder-only language model architectures. Encoder-only models transform input text into contextual

representations and are mainly suited for understanding tasks such as classification or semantic analysis. Encoder-decoder models use an encoder to represent an input sequence and a decoder to generate an output sequence, making them suitable for sequence-to-sequence tasks such as translation and summarization. Decoder-only models, which are especially important in current generative LLMs, generate text autoregressively by predicting the next token from the preceding context [3]. Because the user’s prompt and all previously generated tokens become the conditioning context, decoder-only models are particularly well aligned with open-ended text generation.

Computational Characteristics of Attention

The strength of self-attention is also the source of one of the major efficiency limitations of Transformer-based LLMs. In the classical Transformer, attention compares every token with every other token in the sequence. Consequently, the attention matrix grows quadratically with the sequence length. The reviewed survey on resource-efficient LLMs characterizes the standard self-attention operation as having time complexity $O(T^2d)$ and memory complexity $O(T^2)$, where T is the sequence length and d is the hidden feature dimension [3]. This quadratic scaling becomes increasingly problematic for long prompts and long-context applications.

The inference-focused surveys identify three major sources of inefficiency: large model size, the quadratic complexity of attention, and the autoregressive decoding strategy [19], [3]. Large model size increases the amount of memory required to store weights and raises the cost of moving those weights through the hardware memory hierarchy. Attention increases computational and memory costs as the context length grows. Autoregressive decoding limits parallelism because each generated token depends on the tokens generated before it. These factors jointly affect latency, throughput, memory usage, energy consumption, and deployment cost [19], [3].

Inference Process of Decoder-Only LLMs

Inference is the phase in which a trained LLM uses fixed model parameters to generate an output for a given prompt. In decoder-only LLMs, inference is based on autoregressive language modeling. The model receives an initial token sequence, estimates a probability distribution for the next token, selects or samples a token from that distribution, appends it to the context, and repeats the procedure until a stopping condition is reached. Thus, generation proceeds sequentially, even though many operations inside a single forward pass are parallelizable [19], [3].

The reviewed papers divide LLM inference into two main stages: the prefilling stage and the decoding stage [19], [3]. During the prefilling stage, the model processes the initial prompt tokens. For each Transformer layer, it computes the query, key, and value representations associated with the prompt. The key and value tensors are stored in a key-value cache, commonly called the KV cache. This stage also produces the first

output token. Since the entire prompt is processed at once, the prefilling stage contains substantial matrix-matrix operations and is strongly affected by the length of the input sequence [19], [3].

During the decoding stage, the model generates one new token at a time. At each step, the newly generated token is embedded and passed through the Transformer layers. Instead of recomputing keys and values for all previous tokens, the model reuses the KV cache created during prefilling and extends it with the key and value vectors of the new token. In a decoder layer, the current query attends over the concatenation of cached keys and newly computed keys, and the resulting attention output is then passed through the remaining submodules. This cache-based procedure substantially reduces redundant computation compared with recomputing the entire attention history at every step [19], [3].

However, the KV cache introduces its own memory cost. As more tokens are generated, the cache grows with the total sequence length. For long contexts or large batch sizes, the KV cache can become a major component of memory consumption [3]. This is particularly important for serving systems, where many user requests may be processed concurrently. In such settings, memory is consumed not only by the model weights but also by the active KV caches of multiple requests. Therefore, peak memory during inference is approximately determined by the combination of model weights, KV cache, and temporary activations [19], [3].

Latency, Throughput, and Memory Bottlenecks

The inference process is evaluated using several performance indicators. The first-token latency measures the time required to process the prompt and produce the first generated token. This metric mainly reflects the cost of the prefilling stage. The per-output-token latency measures the average time required to generate each subsequent token during decoding. The total generation latency measures the time required to generate the complete output sequence. Throughput can be measured either as generated tokens per second or completed requests per second [19].

The two inference stages differ in their computational behavior. According to the Roofline-based analysis, the prefilling stage often involves larger matrix-matrix multiplications and may become compute-bound, especially for long input sequences. In contrast, the decoding stage is often memory-bound because each step processes only one new token but must repeatedly load large model weights and access the KV cache [3]. This distinction is important because optimization strategies must match the bottleneck. If computation is the limiting factor, low-precision arithmetic or more efficient kernels may improve performance. If memory bandwidth is the limiting factor, reducing parameter size, reducing memory access, improving cache management, or increasing batching efficiency may be more effective [3].

The inference surveys therefore classify efficiency methods according to the level at which

they intervene. Data-level methods reduce the amount of input or reorganize output generation, for example by compressing prompts or enabling more parallel generation structures. Model-level methods modify the architecture or representation of the model, for example through efficient attention designs, multi-query or grouped-query attention, quantization, pruning, low-rank factorization, or knowledge distillation. System-level methods optimize inference engines and serving systems through batching, scheduling, memory management, offloading, operator fusion, and distributed execution [19], [3].

Efficiency-Oriented Architectural and Inference Optimizations

The reviewed papers emphasize that improving LLM inference requires addressing both the Transformer architecture and the deployment process. At the architectural level, efficient attention variants aim to reduce the cost of the standard quadratic attention mechanism. Approximate attention methods, kernel-based attention, low-rank attention, sparse attention, and hardware-aware attention attempt to reduce either the number of attention computations or the memory traffic associated with attention [3], [19]. Other architectural alternatives, including state-space models, recurrent-like formulations, long-convolution models, and mixture-of-experts designs, are discussed as attempts to preserve modeling capacity while reducing inference cost or improving scalability [3], [19].

At the model-compression level, quantization is particularly important. It reduces storage and memory traffic by representing weights, activations, or KV-cache entries with lower precision. Post-training quantization is attractive because it can be applied after training without fully retraining the model, while quantization-aware training and quantization-aware fine-tuning attempt to preserve accuracy under lower-precision representations [19], [3]. Pruning removes weights, neurons, layers, or other structures considered less important. Structured pruning is usually more hardware-friendly because it removes regular components, whereas unstructured pruning may preserve accuracy but often requires specialized sparse computation support [3], [3]. Low-rank factorization approximates large weight matrices using smaller matrix products, thereby reducing memory usage and the amount of parameter data loaded during decoding [19]. Knowledge distillation transfers behavior or internal knowledge from a larger teacher model to a smaller student model, reducing inference cost by deploying the smaller model [19], [3].

System-level optimization is equally central because LLM inference is not only an algorithmic problem but also a hardware and serving problem. The Roofline analysis highlights that the same model operation may be compute-bound or memory-bound depending on batch size, sequence length, hardware bandwidth, and numerical precision [3]. Consequently, practical inference systems must manage memory allocation, KV-cache storage, request batching, scheduling, and operator execution carefully. Techniques such as FlashAttention, PagedAttention, offloading, and operator fusion are representative of this system-oriented direction, because they improve the way attention and model execution interact with GPU memory and compute resources [3], [19], [3].

Summary

In summary, the three papers present Transformer-based LLMs as systems whose effectiveness arises from stacked self-attention and feed-forward layers, but whose deployment difficulty arises from the same design choices that make them powerful. Token embeddings and positional information provide the initial representation, multi-head self-attention constructs contextual dependencies, feed-forward networks transform token-wise features, and decoder-only autoregressive prediction enables flexible text generation. During inference, the process separates into prefilling and decoding. Prefilling processes the prompt and initializes the KV cache, while decoding repeatedly generates one token, updates the cache, and conditions the next prediction on the expanded context. The principal efficiency challenges are large model size, quadratic attention cost, limited parallelism in autoregressive decoding, and memory traffic associated with model weights and KV caches. The surveyed optimization methods therefore target data, model, system, and hardware levels, aiming to reduce computation, memory usage, memory access, latency, energy consumption, and monetary cost while preserving the quality of generated language.

State-of-the-art LLMs are decoder-only Transformers [18]: a learned embedding layer, L stacked identical Transformer blocks, and a language modelling head. We briefly introduce each component; Chapter ?? then traces how they combine to form the inference pipeline and where efficiency bottlenecks arise.

Tokenisation and embedding. A tokeniser (e.g. Byte-Pair Encoding) converts raw text into a sequence of integer token IDs drawn from a vocabulary of size $|V|$ (typically 32 000–128 000). Each ID indexes a learned embedding matrix $\mathbf{W}_E \in \mathbb{R}^{|V| \times d}$ to produce a dense vector in $\mathbb{R}^d$; for a 7B-parameter model $d = 4\,096$. Rotary Position Embedding (Rotary Positional Encoding (RoPE)) [18] is then added element-wise to inject token-order information without expanding the hidden dimension.

Multi-Head Attention. Each Transformer block applies Multi-Head Attention (MHA) to its input $\mathbf{X} \in \mathbb{R}^{n \times d}$. For each of h heads, projection matrices $\mathbf{W}_i^Q, \mathbf{W}_i^K, \mathbf{W}_i^V \in \mathbb{R}^{d \times d_k}$, with $d_k = d/h$, produce query, key, and value matrices; scaled dot-product attention is then computed:

$$\text{Attention}(\mathbf{Q}_i, \mathbf{K}_i, \mathbf{V}_i) = \text{softmax}\left(\frac{\mathbf{Q}_i \mathbf{K}_i^\top}{\sqrt{d_k}}\right) \mathbf{V}_i. \quad (2.1.2)$$

The outputs of all heads are concatenated and projected by $\mathbf{W}^O \in \mathbb{R}^{hd_k \times d}$. A causal mask enforces that each position t attends only to positions $\leq t$, enabling auto-regressive generation. The term $\mathbf{Q}_i \mathbf{K}_i^\top \in \mathbb{R}^{n \times n}$ is the root of attention’s $\mathcal{O}(n^2)$ cost, which dominates long-context prefill.

Feed-Forward Network. After attention, each position is processed independently by a two-layer Multilayer Perceptron (MLP):

$$\text{FFN}(\mathbf{x}) = \sigma(\mathbf{x}\mathbf{W}_1)\mathbf{W}_2, \quad (2.1.3)$$

where $\mathbf{W}_1 \in \mathbb{R}^{d \times d_{\text{ff}}}$ and $\mathbf{W}_2 \in \mathbb{R}^{d_{\text{ff}} \times d}$. The canonical expansion ratio $d_{\text{ff}} = 4d$ yields $8d^2$ parameters per layer. Modern architectures [17] use the gated SwiGLU variant, $\text{FFN}(\mathbf{x}) = (\sigma(\mathbf{x}\mathbf{W}_g) \odot \mathbf{x}\mathbf{W}_1)\mathbf{W}_2$, with $d_{\text{ff}} \approx \frac{8}{3}d$. Across a full model the Feed-Forward Network (FFN) sub-layers account for approximately 65% of all parameters and constitute the majority of the memory-bandwidth load during decoding.

Language modelling head. After the final block, $\mathbf{W}_{LM} \in \mathbb{R}^{d \times |V|}$ (weight-tied with $\mathbf{W}_E$) maps the last hidden state to a logit vector over the vocabulary. Softmax converts logits to probabilities; greedy, temperature, top- k , or nucleus (top- p) sampling selects the next token.

2.2 The Autoregressive Inference Pipeline

Decoder-only LLMs generate text *autoregressively*: each output token is conditioned on all preceding tokens, forcing a sequential loop. Inference divides into two temporally distinct phases with fundamentally different resource profiles.

Prefill phase. All n_{prompt} input tokens are processed simultaneously in a single parallelised forward pass. The $\mathcal{O}(n^2d)$ attention matrix product dominates; arithmetic intensity (operations per byte of memory traffic) is high, so this phase is *compute-bound*. Its output is the first generated token and a fully populated KV cache.

KV cache. To avoid recomputing attention keys and values at every subsequent decode step, the key and value tensors produced by each layer during prefill are stored in a *KV cache*. For a model with L layers, h heads of dimension d_k , and a sequence of current length n , the cache occupies

$$\text{KV-cache} = 2 \cdot L \cdot h \cdot d_k \cdot n \cdot b_{\text{elem}} \text{ bytes}, \quad (2.2.1)$$

where b_{elem} is bytes per element (2 for FP16). For LLaMA-2-7B ($L = 32$, $h = 32$, $d_k = 128$) this amounts to ≈ 0.5 MB per token; at 4096 tokens the cache reaches ≈ 2 GB—one-sixth of the model weights. For long contexts or large batches the cache can exceed the model weights, making it the primary obstacle to edge deployment.

Decode phase. Each decoding step processes a single new token. The attention reduces to an $\mathcal{O}(nd_k)$ vector–matrix product per head, but all P weight matrices and the accumulated KV cache must be streamed from off-chip DRAM into the compute unit for every token. Because arithmetic work per byte loaded is $\mathcal{O}(1)$ at batch size 1, the decode phase is *memory-bandwidth-bound*: its throughput is capped by DRAM bandwidth, not peak FLOP/s [15]. This asymmetry—compute-bound prefill, bandwidth-bound decode—motivates the distinct optimisation strategies surveyed in Chapter chapter 3.

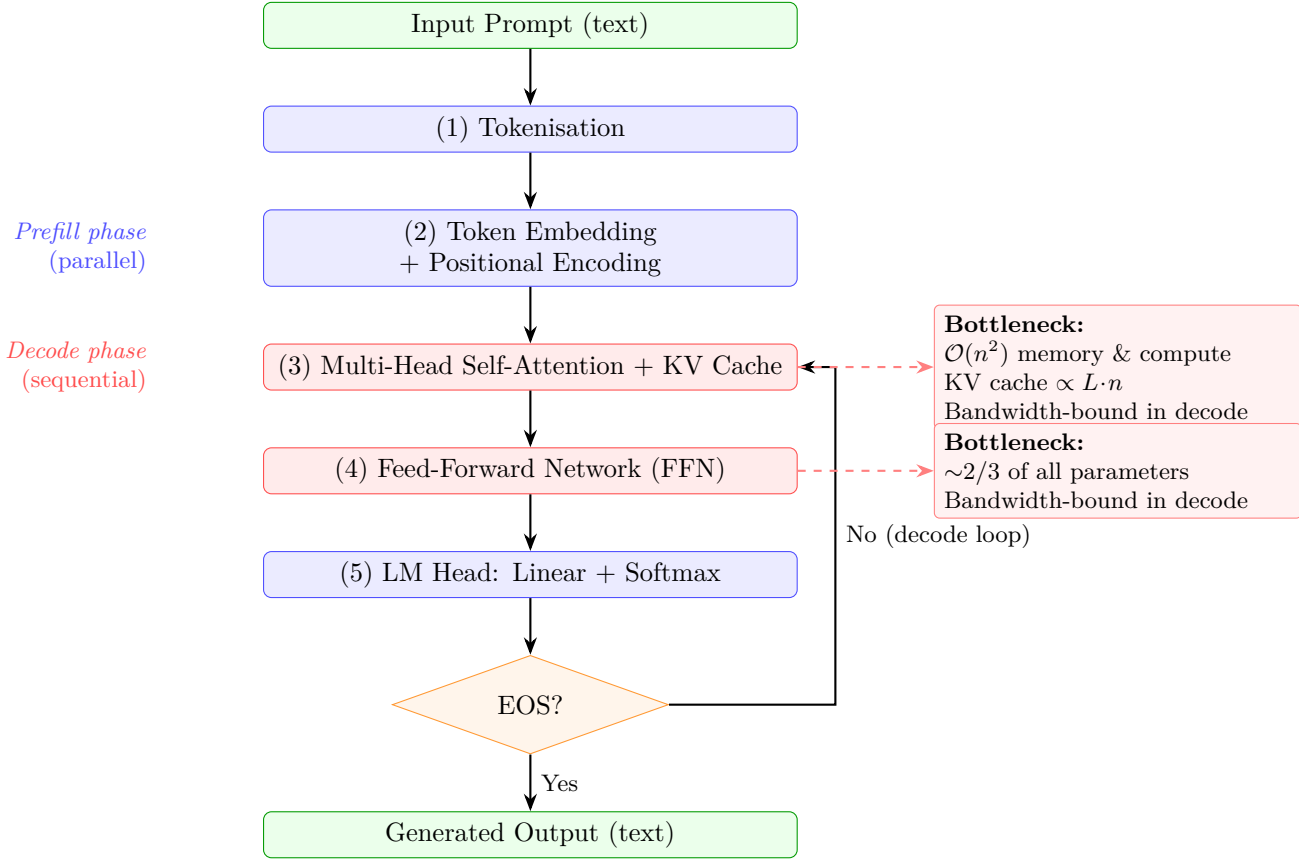


Figure 2.1: Schematic of the Transformer inference pipeline. Red-shaded steps (3, 4) represent the primary resource bottlenecks. The decode loop repeats steps 3–5 until an end-of-sequence token is produced.

2.3 Step-by-Step Bottleneck Analysis

2.3.1 Step 1: Tokenisation

The tokeniser maps a string of characters to a sequence of integer IDs using a fixed vocabulary and a pre-trained merge table (e.g., Byte-Pair Encoding). This step is computationally negligible. Its resource relevance is indirect: a poorly chosen tokenisation scheme may increase the average sequence length n , which in turn scales the KV cache and the cost of the attention computation.

2.3.2 Step 2: Token Embedding and Positional Encoding

The embedding lookup is a simple table indexing operation: $O(n \cdot d_{\text{model}})$ memory reads with no arithmetic. For large vocabularies (e.g., 128 000 tokens) the embedding matrix $\mathbf{W}_E$ can occupy several hundred megabytes. Because the embedding table is accessed irregularly (depending on which tokens appear in the prompt), it exercises memory bandwidth with poor cache locality.

2.3.3 Step 3: Multi-Head Self-Attention

The attention sub-layer is the most analytically interesting bottleneck. Its resource costs arise from two sources:

Arithmetic cost. Computing $\mathbf{QK}^\top$ for a sequence of length n requires $O(n^2 d_k)$ floating-point operations, giving quadratic growth in sequence length. During prefill this dominates for long contexts. During decode, $n_{\text{new}} = 1$, so the arithmetic cost collapses to $O(nd_k)$ —a single dot-product per head per past token.

KV cache memory. To avoid recomputing keys and values for every past token, the model caches them after prefill. As derived in Section 2.2, this cache can equal or exceed the model weights in size for long sequences. Its growth with sequence length is the principal obstacle to serving many concurrent long-context requests, and to fitting the inference workload on an edge device.

2.3.4 Step 4: Feed-Forward Network

The FFN sub-layer applies two (or three, in gated architectures) large matrix multiplications per layer. For a standard configuration, $\mathbf{W}_1 \in \mathbb{R}^{d_{\text{model}} \times 4d_{\text{model}}}$ and $\mathbf{W}_2 \in \mathbb{R}^{4d_{\text{model}} \times d_{\text{model}}}$, so the FFN accounts for $8d_{\text{model}}^2$ parameters per layer—approximately twice the parameter count of the attention projection matrices. Across all L layers, the FFN weights represent roughly 65% of total model parameters.

During prefill, the FFN is compute-bound: the large matrix multiplications achieve high Model FLOP Utilisation (MFU). During decode (single-token input), the same weight matrices must be read from Dynamic Random Access Memory (DRAM) but perform only $O(d_{\text{model}})$ arithmetic operations per column—making this phase heavily bandwidth-bound.

2.3.5 Step 5: LM Head and Sampling

The LM head projects the final hidden state to a logit vector of size $|V|$. This is another large matrix multiplication ($\mathbb{R}^{d_{\text{model}} \times |V|}$, which is often shared with $\mathbf{W}_E$ to save memory). The subsequent softmax and sampling step are computationally inexpensive but introduce a sequential dependency: the next token cannot be determined until sampling is complete, enforcing a fundamental serialisation in standard autoregressive decoding.

2.4 Limitations Beyond Inference

Several of the bottlenecks described above are not exclusive to the inference phase. This distinction is relevant when evaluating optimisation strategies.

Model weight size. The memory footprint of the weights is identical whether the model is being trained or served. Compression techniques such as quantisation and pruning that reduce weight memory therefore benefit both phases. Their costs—accuracy degradation and potential increases in calibration compute—are also shared.

KV cache growth. The KV cache is an inference-time construct: it does not exist during training. However, its size is a direct consequence of architectural decisions made before training (number of layers L , number of attention heads h , head dimension d_k), which cannot be changed post hoc without retraining. Architectural choices such as Grouped-Query Attention (GQA) [2] or Multi-Query Attention (MQA) [16] that reduce the per-layer KV footprint must therefore be baked into the model from the outset.

Arithmetic intensity. The bandwidth-vs-compute characterisation of decode-phase inference stems from the small batch size of autoregressive generation (effective batch size = 1 per concurrent request). Techniques such as continuous batching, which increase the effective batch size by grouping multiple requests, can shift the operating point towards the compute-bound regime—but require system-level changes and are primarily applicable in server settings.

2.5 Taxonomy and Key Metrics

The following terms appear consistently throughout this paper.

KV cache. The stored key and value tensors for all past tokens, maintained to avoid recomputation at each decode step (Equation (2.2.1)).

Quantisation. Representing weights and/or activations at reduced numerical precision—e.g. FP16 $\rightarrow$ INT8 or INT4—to reduce memory footprint and, on hardware with native low-precision arithmetic, increase throughput.

Pruning. Removing parameters from a trained model. *Unstructured* pruning zeros individual weights; *structured* pruning removes entire rows, columns, heads, or layers to yield a dense, smaller model directly executable on standard hardware.

Knowledge Distillation. Training a compact *student* model to match the output distribution of a larger *teacher*, transferring capability at a fraction of the inference cost.

Throughput. Tokens (or requests) completed per second across all concurrent users; the primary efficiency metric for server-side deployment.

Latency. Elapsed time from request arrival to final-token delivery. Time to First Token (TTFT) (time to first token) governs perceived responsiveness; Time per Output Token (TPOT) (time per output token) governs generation speed.

Arithmetic intensity. Ratio of floating-point operations to bytes of memory traffic, in OPs/byte. Kernels below the hardware ridge point are bandwidth-bound; those above are compute-bound.

Edge device. Any computing platform with constrained memory, power, and compute operating outside a data centre: smartphones, embedded controllers, and automotive compute units.

3 Towards Efficient Inference: Techniques and Solutions

Drawing on the bottleneck analysis of ??, this chapter systematically surveys the techniques that have been developed to reduce the resource demands of LLM inference. For each technique the discussion covers its core mechanism, the bottleneck it addresses, its accuracy–efficiency trade-off, and its practical implications for edge deployment. Figure figure 3.1 provides an overview of how the techniques presented in this chapter map to the bottlenecks identified in ??.

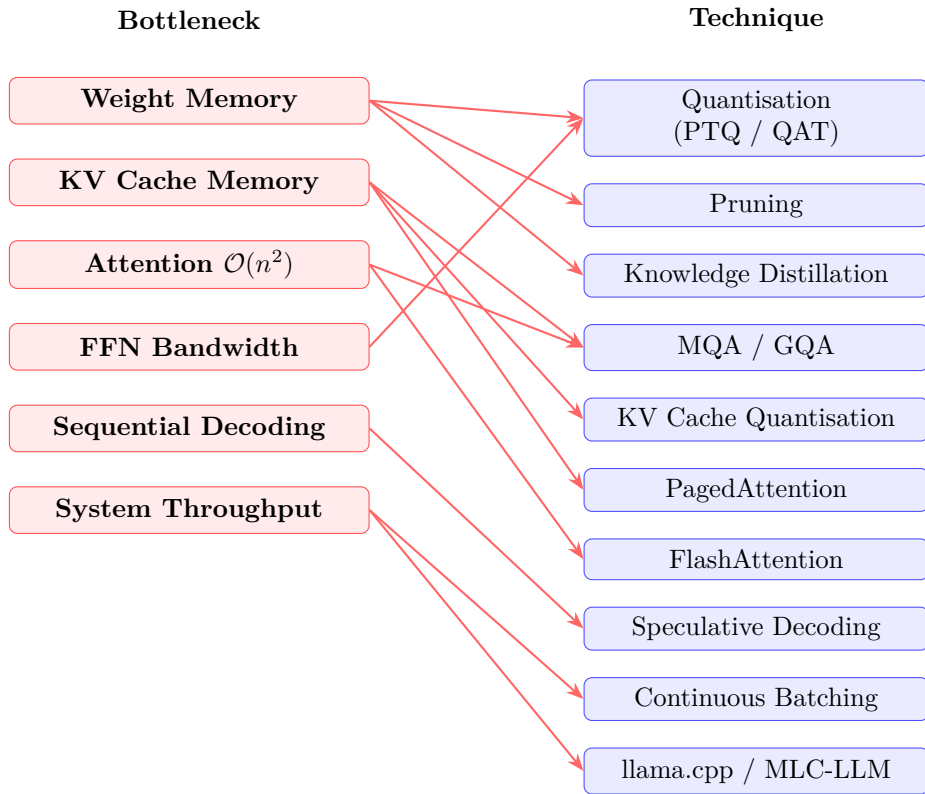


Figure 3.1: Mapping of inference bottlenecks (red) to the efficiency techniques (blue) surveyed in this chapter. Arrows indicate which technique targets which bottleneck; several techniques address multiple bottlenecks simultaneously.

3.1 Model Compression

Model compression techniques reduce the size of the model itself—shrinking weight memory and, consequently, the bandwidth required to load those weights during each decode step. They are pre-deployment operations and impose a one-time cost in compute or data for calibration.

3.1.1 Quantisation

Quantisation is the process of representing the floating-point weights (and optionally activations) of a trained model at reduced numerical precision. It is currently the most widely adopted compression technique for edge LLM deployment because it yields large memory reductions with acceptable quality loss and requires no architectural changes.

Why quantisation helps. A weight stored in 16-bit Floating Point (FP16) occupies 2 bytes. Representing it instead in 8-bit integer format (8-bit Integer (INT8)) halves weight memory and doubles arithmetic throughput on hardware that natively supports INT8 matrix multiply (e.g., NVIDIA Tensor Cores, ARM Cortex-A78 NEON). Reducing to 4-bit integers (4-bit Integer (INT4)) achieves a $4\times$ compression relative to FP16, enabling a 7B model to fit in as little as 3.5 GB of memory.

Post-Training Quantisation (PTQ). In Post-Training Quantisation (PTQ), a fully trained model is quantised without further gradient-based updates. The simplest approach, round-to-nearest (RTN), maps each weight w to the nearest representable integer value; it is fast but introduces quantisation error that can significantly degrade perplexity below 4 bits.

More sophisticated PTQ methods achieve substantially better accuracy. **GPTQ** [8] applies a second-order optimisation using the inverse Hessian of the layer-wise reconstruction error to determine quantisation order and compensate remaining weights for the quantisation error of previously quantised weights. This allows accurate weight-only quantisation to INT4 with less than 1 point of perplexity increase on LLaMA-class models.

AWQ [13] takes a complementary perspective: it observes that the importance of a weight is correlated with the magnitude of the corresponding activation, and that protecting a small fraction (1%) of salient weights in higher precision (while quantising the rest to INT4) preserves most of the model quality. AWQ is *activation-aware* in the sense that it uses a small calibration dataset to identify these salient channels before quantisation.

LLM.int8() [7] addresses quantisation from the activation side. It observes that LLM

activations contain a small number of outlier features with magnitudes $100\times$ larger than the mean. Quantising these outliers causes disproportionate error. The solution is a mixed-precision decomposition: outlier feature dimensions are kept in FP16 while the remaining $\sim 99.9\%$ are multiplied in INT8, with no loss in model quality at INT8 precision on models above 6.7B parameters.

Quantisation-Aware Training (QAT). In Quantisation-Aware Training (QAT), the quantisation operation is simulated during training by inserting ‘fake quantisation’ nodes that round activations and weights during the forward pass but compute gradients through the rounding operation via the straight-through estimator. The model therefore learns to be robust to quantisation noise. QAT typically achieves better accuracy than PTQ at very low bit widths (2–4 bits), but requires re-training access to the original data and substantial compute—making it less practical when a model’s training data is proprietary.

Edge implications. Quantisation is the *de facto* entry point for edge LLM deployment. Tools such as llama.cpp (see Section section 3.4) implement custom 2–8 bit integer formats (GGUF quantisation) specifically optimised for ARM and x86 CPUs, enabling consumer hardware to run 7B and 13B models interactively. The dominant trade-off is quality vs. compression ratio: below 4 bits, perplexity degradation becomes increasingly noticeable, and the optimal bit-width is task-dependent [3].

3.1.2 Pruning

Pruning removes parameters from the model that contribute least to its predictions, reducing both weight memory and Floating Point Operation (FLOP) count. The two principal strategies—*unstructured* and *structured* pruning—differ in *what* they remove and, consequently, in the hardware speedup they deliver. Figure ?? illustrates both approaches on a concrete 4×4 weight matrix.

Unstructured pruning zeros individual weights according to a saliency criterion; the most widely used is *magnitude pruning*: weight w_{ij} is pruned if $|w_{ij}| < \theta$ for a chosen threshold θ . As illustrated in Figure ??(b), applying $\theta = 0.25$ to $\mathbf{W}$ zeros seven of the sixteen entries (44% sparsity) while retaining the nine larger-magnitude weights intact. The critical limitation is that the resulting sparse matrix is *structurally identical* to the original: non-zero values follow no regular pattern, so the data must be encoded in a sparse format (e.g., Compressed Sparse Row), and standard dense Basic Linear Algebra Subprograms (BLAS) routines cannot skip the zeroed positions. Practical speedups therefore require hardware support for sparse arithmetic—available on NVIDIA Ampere Graphics Processing Units (GPUs) via 2:4 structured sparsity but largely absent from edge Central Processing Units

(CPUs). Despite this constraint, high sparsity levels (80–90%) can be achieved with minimal quality loss for moderately sized models [3], making unstructured pruning attractive when the inference platform supports sparse matrix operations.

Structured pruning removes entire, regular sub-structures—rows or columns of weight matrices, attention heads, FFN intermediate dimensions, or whole Transformer layers—yielding a strictly smaller *dense* matrix that requires no sparse encoding and executes efficiently on any standard hardware. As shown in Figure ??(c), the row with the smallest ℓ_2 norm ($\mathbf{r}_1$, norm ≈ 0.78) is eliminated entirely: the 4×4 matrix shrinks to a 3×4 matrix, delivering a genuine 25% reduction in both parameter count and multiply–accumulate operations with no specialised kernel required.

LLM-Pruner [14] automates structured pruning of LLMs by analysing the dependencies between coupled weight groups (e.g., attention heads and their corresponding projection matrices) and applying gradient-based importance estimation to select which groups to remove. After pruning, a brief post-training or Low-Rank Adaptation (LoRA)-based fine-tuning step recovers accuracy. In experiments on LLaMA, LLM-Pruner achieves a 20% parameter reduction with less than 3 points perplexity degradation and no access to the original pre-training data.

Head pruning specifically targets attention heads. Studies have shown that many attention heads learn redundant or near-uniform patterns and can be removed without quality loss [3]. Removing k heads out of h per layer reduces the attention KV footprint proportionally, providing a complementary benefit to techniques that reduce KV cache size.

Layer pruning removes entire Transformer blocks. Because blocks are applied sequentially, removing a block reduces latency proportionally. Empirical results on LLaMA-2 show that removing up to 25% of layers causes less than 5% perplexity increase on standard benchmarks; the degradation is larger for tasks requiring deep reasoning chains.

Edge implications. Structured pruning is particularly attractive for edge deployment because the resulting model is simply a smaller version of the original and requires no specialised runtime. A pruned-then-quantised 7B model can realistically be served on a high-end smartphone: pruning to a 5B effective parameter count followed by INT4 quantisation reduces memory from 14 GB to approximately 2.5 GB.

3.1.3 Knowledge Distillation

Knowledge distillation (KD) [10] trains a smaller *student* model to mimic the behaviour of a larger *teacher* model. Rather than training the student on hard labels (the ground-truth next token), the student is trained to match the teacher’s soft probability distribution over the vocabulary. Because the teacher’s output distribution encodes information about semantically related tokens—including near-misses and plausible alternatives—the student receives a richer training signal than hard-label supervision.

Task-specific distillation fine-tunes a small student on a task-specific corpus using the teacher’s outputs as supervision. This produces compact models optimised for narrow applications (e.g., a 1B model for a specific domain), but the student does not generalise beyond the training distribution.

Task-agnostic / general distillation trains a student from scratch (or from a pre-trained checkpoint) to mimic the pre-training distribution of the teacher. This is far more expensive (requiring access to large corpora and significant compute) but produces a student that retains general-purpose capabilities.

White-box vs. black-box distillation. When the teacher’s internal representations are accessible (*white-box*), the student can be trained to match not only the output logits but also intermediate hidden states and attention maps, providing additional supervision signal. When only the teacher’s output text is available (*black-box*), the student is trained on the teacher-generated text as a dataset.

Edge implications. Distillation is the only compression technique that does not start from a large model: it produces a small model from the outset. For edge deployment, 1B and 3B-parameter models distilled from 70B+ teachers (e.g., Phi-2, Phi-3 from Microsoft) have demonstrated that compact architectures can retain a large fraction of the teacher’s reasoning capability when the distillation corpus is carefully curated.

3.2 Inference-Level Optimisations

These techniques modify the attention mechanism or the KV cache management strategy without altering the model weights. They can be applied at inference time, often transparently to the model.

3.2.1 Multi-Query and Grouped-Query Attention

Standard MHA maintains one independent set of key and value projection matrices per head. With h heads, the KV cache therefore stores h distinct key and value tensors per layer and per token, with total per-token cache size $2Lhd_k$.

MQA [16] proposes that all query heads share a single key head and a single value head. The KV cache therefore collapses to $2L \cdot 1 \cdot d_k$ per token—a factor-of- h reduction. Because the KV cache is the second largest contributor to memory consumption after model weights, this reduction is highly significant at edge scale. The trade-off is a slight decrease in model quality (approximately 0.5–1% on downstream benchmarks), which is generally acceptable [16].

GQA [2] generalises MQA by grouping the h query heads into g groups, each sharing one key/value head. With $g = h$ this reduces to standard MHA; with $g = 1$ to MQA. In practice, $g = h/4$ to $h/8$ recovers most of the MHA quality while approaching MQA efficiency. GQA has been adopted in LLaMA-2 (70B), LLaMA-3, and Mistral, indicating that the field has largely converged on this as the standard attention variant.

Edge implications. Since MQA/GQA must be incorporated during pre-training, they are not applicable to already-trained models. However, GQA can be retrofitted via a brief fine-tuning step (uptraining from MHA checkpoints) at a fraction of the original training cost [2]. For new models targeting edge deployment, GQA should be considered mandatory.

3.2.2 FlashAttention

The standard MHA implementation loads the full $n \times n$ attention score matrix into High-Bandwidth Memory (HBM) before applying the softmax and value aggregation. For a sequence length of $n = 4096$ tokens with FP16 precision, this matrix occupies $n^2 \times 2$ bytes = 32 MB per head. With 32 heads, this is 1 GB of temporary HBM traffic per layer—a dominant bandwidth cost.

FlashAttention [6] reformulates the attention computation to avoid materialising the full attention matrix. It tiles the input matrices into blocks that fit entirely in the GPU’s fast on-chip Static Random Access Memory (SRAM), and fuses the computation of $\mathbf{QK}^\top$, softmax, and $\text{softmax}(\cdot)\mathbf{V}$ into a single kernel. This eliminates the round-trip to HBM for the intermediate attention matrix, reducing HBM accesses from $O(n^2)$ to $O(n)$ while computing the numerically identical result (it is an exact, not approximate, algorithm).

Practically, FlashAttention achieves 2–4 $\times$ speedup on attention computation relative to the naive implementation, with proportionally lower energy consumption—making it directly beneficial for edge inference on any hardware with a fast cache hierarchy.

3.2.3 KV Cache Optimisation

PagedAttention. The KV cache grows dynamically with sequence length. In the standard contiguous allocation scheme, each new request pre-allocates a fixed-size maximum-length block of KV memory, leading to severe internal fragmentation when requests terminate early. Across thousands of concurrent requests on a server, this wastage can reduce effective throughput by 60–80% [11].

PagedAttention [11], implemented in the vLLM framework, adapts the virtual memory paging concept to the KV cache. Each request’s KV vectors are stored in fixed-size *pages* that need not be physically contiguous. A page table maps logical to physical page addresses at inference time. This eliminates nearly all internal and external fragmentation, increasing the effective number of concurrent requests that can be served from the same GPU memory by 2–4 $\times$.

KV cache quantisation. The KV cache can itself be stored at reduced precision. Quantising the cached key/value tensors to INT8 halves cache memory with minimal perplexity impact. More aggressive quantisation to INT4 is possible with careful calibration, providing a further 2 $\times$ reduction.

KV cache eviction. For very long sequences, it is possible to *evict* (discard) cache entries for tokens that are unlikely to be attended to by future queries, based on heuristics such as attention score recency or magnitude. StreamingLLM and similar methods maintain a fixed-size cache window, enabling effectively unbounded context length at the cost of some context fidelity.

3.3 Decoding Strategies

3.3.1 Speculative Decoding

Standard autoregressive decoding generates one token per forward pass, enforcing a strict latency lower bound of (number of generated tokens) $\times$ (time per forward pass). This

is wasteful when the full model is large but most token generations are ‘easy’ (highly predictable).

Speculative decoding [12] exploits this insight with a two-model approach:

1. A small, fast *draft model* (e.g., 1–3B parameters) generates K candidate tokens autoregressively in K forward passes.
2. The large *target model* processes all K draft tokens in a single forward pass (which is parallelisable, like prefill) and accepts or rejects each draft token using a rejection sampling criterion that guarantees the accepted sequence has the same distribution as the target model’s output.

When the draft model’s predictions match the target with probability α (the *acceptance rate*), the expected number of tokens produced per target-model forward pass is $\frac{1-\alpha^{K+1}}{1-\alpha}$. For $\alpha \approx 0.8$ and $K = 5$, this yields approximately 3.5 tokens per call to the target model—a $3.5\times$ latency improvement with no loss in output quality [12].

For edge deployment, the ‘large model’ and ‘draft model’ may themselves be a quantised medium model and a tiny model respectively. The framework is flexible: any two models with matching tokenisers and vocabulary can be paired.

3.3.2 Continuous Batching

Traditional static batching waits until a fixed number of requests arrive and then processes them together. Requests of different lengths must be padded to a common length (wasting compute) and the entire batch must finish before new requests are admitted (increasing average latency).

Continuous (or iteration-level) batching inserts new requests into the running batch at the earliest available slot (e.g., whenever a request produces an EOS token and vacates its slot). This keeps hardware utilisation high and reduces median latency significantly. Combined with PagedAttention, continuous batching enables high-throughput serving that is relevant for cloud deployments and, increasingly, for shared edge inference servers in enterprise settings.

3.4 System-Level Runtimes and Edge Frameworks

Efficiency techniques are only realised in practice when they are implemented in well-engineered software runtimes that expose hardware capabilities efficiently. Several frame-

works have emerged specifically for LLM inference, with differing strengths on server and edge hardware.

3.4.1 llama.cpp

`llama.cpp` [9] is a C/C++ inference runtime designed for deployment on commodity hardware, including ARM CPUs (Apple Silicon, Android), x86 CPUs, and consumer GPUs. Its key design decisions include:

- **GGUF quantisation format:** a family of mixed-precision weight quantisation schemes (2–8 bits per weight) with hand-tuned dequantisation kernels for NEON (ARM SIMD), AVX2, and CUDA.
- **Minimal dependencies:** the library has no deep learning framework dependency, making it straightforward to compile and deploy on embedded targets.
- **Metal / Vulkan backend:** hardware-accelerated compute on Apple Neural Engine and Mali/Adreno GPUs on mobile devices.

In practice, `llama.cpp` enables a quantised (Q4_K_M) 7B model to generate tokens at 20–40 tokens/second on a modern laptop CPU and at 60–80 tokens/second on an Apple M-series chip—sufficient for interactive use [1].

3.4.2 MLC-LLM

MLC-LLM [5] (Machine Learning Compilation (MLC) Large Language Model) takes a compilation-based approach. It uses the Apache TVM deep learning compiler to generate hardware-specific compute kernels from a high-level model description, targeting iOS (Metal), Android (OpenCL, Vulkan), WebGPU, CUDA, and ROCm from a single code base. Key advantages include:

- Auto-tuning of tiling and scheduling parameters for the target hardware.
- Support for both weight quantisation and KV cache quantisation.
- On-device inference on smartphones (e.g., LLaMA-2 7B at 3–8 tokens/second on a Pixel 7 Pro).

The compilation step adds several minutes of one-time overhead but produces deployment artefacts that run efficiently with no runtime compilation cost.

3.4.3 vLLM

vLLM [11] is optimised for *server-side* high-throughput serving. Its core contribution is PagedAttention (Section section 3.2.3), which dramatically reduces KV cache memory fragmentation. vLLM additionally implements continuous batching, FlashAttention, and quantised inference, providing a production-grade serving stack for data-centre deployment. While not directly targeted at edge hardware, vLLM establishes the baseline for cloud-side efficient inference from which edge-focused frameworks draw inspiration.

3.5 Discussion: Trade-offs and Edge-Specific Considerations

No single technique dominates across all deployment scenarios. The efficiency gains and quality losses described above interact in ways that must be understood holistically when targeting edge hardware.

3.5.1 The Efficiency–Accuracy Trade-off

Compression techniques invariably introduce some quality degradation. The magnitude of this degradation depends on:

- **Bit width:** quantisation from FP16 to INT8 is nearly lossless; from INT8 to INT4 introduces a noticeable but usually acceptable quality reduction; below INT4, degradation accelerates [8].
- **Task sensitivity:** arithmetic, logical reasoning, and mathematical tasks degrade faster under quantisation than open-ended text generation [3].
- **Model scale:** larger models exhibit greater robustness to compression. A 70B model quantised to INT4 often outperforms an uncompressed 7B model, making the question ‘which compressed large model vs. which uncompressed small model’ a central practical question for edge deployment.

3.5.2 Orthogonality and Composability

Many of the techniques described in this chapter are *orthogonal*: they address different bottlenecks and can be applied simultaneously. A practical edge stack might combine:

1. **Distillation** (at training time) to obtain a smaller base model.
2. **GQA** (in the model architecture) to reduce KV cache size.
3. **GPTQ or AWQ quantisation** (post-training) to compress weights to INT4.
4. **FlashAttention** (at inference time) to reduce attention bandwidth.
5. **Speculative decoding** (at inference time) to improve throughput.
6. **llama.cpp or MLC-LLM** as the runtime to execute efficiently on ARM/mobile hardware.

Applied together, these techniques have enabled 7B-class models to be deployed interactively on smartphones in 2024—an outcome that would have seemed impractical just two years earlier.

3.5.3 The Fundamental Edge Constraint: Memory

Across all dimensions, memory remains the hardest constraint on edge devices. Compute and energy can often be traded against latency by reducing clock frequency; bandwidth can be partially mitigated by caching. But there is no recourse when a model simply does not fit in the available DRAM: inference is not possible. This makes weight quantisation and architectural choices that reduce the KV cache (such as GQA) the most impactful interventions for edge deployment, not as performance optimisations but as enabling technologies [1].

4 Conclusion

This paper has traced the resource challenges of LLM inference from first principles through to practical deployment solutions, with a sustained focus on edge hardware as the most demanding deployment target.

Chapter chapter 1 established that the four resource dimensions of LLM inference—compute, memory, memory bandwidth, and energy—are not equally constraining in all settings. On edge devices, memory capacity is the hard, non-negotiable limit: a model that does not fit in DRAM simply cannot run. Memory bandwidth is the dominant performance bottleneck during the decode phase, and energy determines the sustained throughput a device can deliver over time.

Chapter ?? analysed the inference pipeline step by step and identified its two most resource-intensive components: the self-attention sub-layer, which incurs $O(n^2)$ compute during prefill and a growing KV cache during decode, and the FFN sub-layer, which accounts for roughly two-thirds of all model parameters and is memory-bandwidth-bound during decode. The chapter also clarified that while the KV cache is an inference-specific structure, its size is determined by architectural choices made at training time—meaning that the efficiency of inference is, in part, a function of how thoughtfully the model was designed.

Chapter chapter 3 surveyed the state-of-the-art techniques for addressing these bottlenecks. Several key findings emerge from this analysis.

Quantisation is the single most impactful intervention for edge deployment.

Reducing weight precision from FP16 to INT4 achieves a $4\times$ compression of model weights, making 7B-class models feasible on consumer hardware with 4–8 GB of memory. Methods such as GPTQ [8] and AWQ [13] demonstrate that this compression can be achieved with minimal perplexity degradation, particularly for models above 7B parameters.

GQA is now essential architecture. The reduction of the KV cache footprint by a factor of h/g (where h is the number of query heads and g is the number of key-value head groups) is one of the most memory-efficient interventions available. It is cost-free in a newly trained model and recoverable at low cost through uptraining. Given the disproportionate role of KV cache in edge memory budgets, GQA should be considered a baseline requirement for any model targeting edge deployment.

FlashAttention makes long-context inference practical. By eliminating the quadratic memory traffic of the attention matrix, FlashAttention [6] substantially reduces both latency and energy consumption for prefill, and makes longer context windows feasible without exceeding on-chip memory limits.

Speculative decoding provides latency improvements without quality loss. For interactive applications—which are the primary use case for edge inference—reducing latency per generated token is more important than maximising throughput. Speculative decoding [12] directly targets this metric and is composable with other techniques.

Dedicated edge runtimes bridge the gap between algorithmic efficiency and hardware performance. llama.cpp [9] and MLC-LLM [5] demonstrate that practical edge inference is achievable with current 7B-class models when quantisation, hardware-specific SIMD kernels, and efficient memory management are integrated in a single, well-engineered stack.

Bibliography

- [1] “A review on edge large language models: Design, execution, and applications,” *ACM Computing Surveys*, 2024. [Online]. Available: <https://dl.acm.org/doi/10.1145/3719664>
- [2] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai, “GQA: Training generalised multi-query transformer models from multi-head checkpoints,” in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023. [Online]. Available: <https://arxiv.org/abs/2305.13245>
- [3] G. Bai, Z. Chai, C. Ling, S. Wang, J. Lu, C. Li, and L. Yao, “Beyond efficiency: A systematic survey of resource-efficient large language models,” 2024. [Online]. Available: <https://arxiv.org/abs/2401.00625>
- [4] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, “Language models are few-shot learners,” *Advances in Neural Information Processing Systems*, vol. 33, pp. 1877–1901, 2020. [Online]. Available: <https://arxiv.org/abs/2005.14165>
- [5] T. Chen *et al.*, “MLC-LLM: Universal LLM deployment engine with ML compilation,” GitHub repository, 2023. [Online]. Available: <https://github.com/mlc-ai/mlc-llm>
- [6] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, “FlashAttention: Fast and memory-efficient exact attention with IO-awareness,” in *Advances in Neural Information Processing Systems*, vol. 35, 2022. [Online]. Available: <https://arxiv.org/abs/2205.14135>
- [7] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, “LLM.int8(): 8-bit matrix multiplication for transformers at scale,” in *Advances in Neural Information Processing Systems*, vol. 35, 2022. [Online]. Available: <https://arxiv.org/abs/2208.07339>
- [8] E. Frantar, S. Ashkboos, T. Hoefer, and D. Alistarh, “GPTQ: Accurate post-training quantization for generative pre-trained transformers,” 2023. [Online]. Available: <https://arxiv.org/abs/2210.17323>

- [9] G. Gerganov and contributors, “llama.cpp: C/C++ port of Meta’s LLaMA model,” GitHub repository, 2023. [Online]. Available: <https://github.com/ggerganov/llama.cpp>
- [10] G. Hinton, O. Vinyals, and J. Dean, “Distilling the knowledge in a neural network,” 2015. [Online]. Available: <https://arxiv.org/abs/1503.02531>
- [11] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with PagedAttention,” in *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023. [Online]. Available: <https://arxiv.org/abs/2309.06180>
- [12] Y. Leviathan, M. Kalman, and Y. Matias, “Fast inference from transformers via speculative decoding,” in *Proceedings of the 40th International Conference on Machine Learning*, 2023. [Online]. Available: <https://arxiv.org/abs/2211.17192>
- [13] J. Lin, J. Tang, H. Tang, S. Yang, X. Dang, and S. Han, “AWQ: Activation-aware weight quantization for LLM compression and acceleration,” in *Proceedings of Machine Learning and Systems*, vol. 6, 2024. [Online]. Available: <https://arxiv.org/abs/2306.00978>
- [14] X. Ma, G. Fang, and X. Wang, “LLM-Pruner: On the structural pruning of large language models,” in *Advances in Neural Information Processing Systems*, vol. 36, 2023. [Online]. Available: <https://arxiv.org/abs/2305.11627>
- [15] R. Pope, S. Douglas, A. Chowdhery, J. Devlin, J. Bradbury, J. Heek, K. Xiao, S. Young, and J. Dean, “Efficiently scaling transformer inference,” in *Proceedings of Machine Learning and Systems*, vol. 5, 2023. [Online]. Available: <https://arxiv.org/abs/2211.05102>
- [16] N. Shazeer, “Fast transformer decoding: One write-head is all you need,” 2019. [Online]. Available: <https://arxiv.org/abs/1911.02150>
- [17] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar *et al.*, “LLaMA: Open and efficient foundation language models,” 2023. [Online]. Available: <https://arxiv.org/abs/2302.13971>
- [18] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017. [Online]. Available: <https://arxiv.org/abs/1706.03762>

- [19] N. X. H. K. F. T. X. J. L. S. e. a. Zhou, Z., “A survey on efficient inference for large language models.” 2024. [Online]. Available: <https://arxiv.org/abs/2404.14294>